MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

A Project Report on

Hawk Sight Ai:- Ai Stock Market Prediction System

Submitted by,

Atharav Barapatre	**(Exam Seat No. 202301040077)**
Nikhil Bodre	**(Exam Seat No. 202301100008)**
Darshan Bhabad	**(Exam Seat No. 202301040169)**
Mandar Ghadage	**(Exam Seat No. 202301100024)**

Guided by,

Mr.Chaitanya Patil

A Report submitted to MIT Academy of Engineering, Alandi(D), Pune, An Autonomous Institute Affiliated to Savitribai Phule Pune University in partial fulfillment of the requirements of

THIRD YEAR BACHELOR OF TECHNOLOGY in

Computer Engineering

Department of Computer Engineering

MIT Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Alandi (D), Pune – 412105

(2026–2027)

MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

CERTIFICATE

It is hereby certified that the work which is being presented in the Third Year Major Project–1 Report entitled **" Hawk Sight Ai:- Ai Stock Market Prediction "**, in partial fulfillment of the requirements for the award of the Bachelor of Technology in Computer Engineeringand submitted to the **Department of Computer Engineering** of **MIT Academy of Engineering, Alandi(D), Pune, Affiliated to Savitribai Phule Pune University (SPPU), Pune**, is an authentic record of work carried out during Academic Year 2026–2027 Semester V, under the supervision of **Mr.Chaitanya Patil**, **Department of Computer Engineering**

Atharav Barapatre	**(Exam Seat No. 202301040077)**
Nikhil Bodre	**(Exam Seat No. 202301100008)**
Darshan Bhabad	**(Exam Seat No. 202301040169)**
Mandar Ghadage	**(Exam Seat No. 202301100024)**

Mr.Chaitanya Patil
Project Advisor

Dr. Kanchan Dhote
Project Coordinator

Dr.Pramod D. Ganjewar
HoD

External Examiner

DECLARATION

We the undersigned solemnly declare that the project report is based on our own work carried out during the course of our study under the supervision of **Mr.Chaitanya Patil**.

We assert the statements made and conclusions drawn are an outcome of our project work. We further certify that

1. The work contained in the report is original and has been done by us under the general supervision of our supervisor.
2. The work has not been submitted to any other Institution for any other degree/diploma/certificate in this Institute/University or any other Institute/University of India or abroad.
3. We have followed the guidelines provided by the Institute in writing the report.
4. Whenever we have used materials (data, theoretical analysis, and text) from other sources, we have given due credit to them in the text of the report and giving their details in the references.

Atharav Barapatre	**(Exam Seat No. 202301040077)**
Nikhil Bodre	**(Exam Seat No. 202301100008)**
Darshan Bhabad	**(Exam Seat No. 202301040169)**
Mandar Ghadage	**(Exam Seat No. 202301100024)**

Abstract

The stock market is characterized by its volatility, non-linearity, and complex dynamic behavior, making accurate price prediction one of the most challenging tasks in financial engineering. Traditional statistical methods often fail to capture the long-term dependencies and recurrent patterns inherent in financial time-series data. Furthermore, retail investors frequently suffer from emotional biases, leading to suboptimal trading decisions.

This project presents "Hawk Sight :-AI Stock Market Prediction," a comprehensive system designed to forecast stock trends and simulate autonomous trading. The primary objective is to develop and compare the performance of three distinct machine learning architectures: **Linear Regression**, **Random Forest Regressor**, and a Deep Learning-based **Long Short-Term Memory (LSTM)** network. The system utilizes historical market data (Open, High, Low, Close, Volume) fetched via the Yahoo Finance API to train these models.

Experimental results demonstrate that the LSTM network significantly outperforms traditional regression and ensemble methods in minimizing the Root Mean Squared Error (RMSE), effectively capturing the temporal sequences of stock prices. Beyond prediction, the project integrates a novel Auto-Buy algorithm linked to a virtual wallet, which executes simulated trades based on generated signals, thereby removing human emotional error. The entire ecosystem is deployed on a user-friendly **Streamlit** web interface, providing real-time visualization of data, technical indicators, and portfolio performance. This work bridges the gap between complex quantitative analysis and accessible, automated financial tools.

Acknowledgment

It gives us great pleasure in presenting the project report on **"AI Stock Market Prediction"**.

We take this opportunity to express our deep sense of gratitude to our project guide, **Mr. Chaitanya Patil**, for his valuable guidance, profound advice, and continuous encouragement throughout the course of this project. His technical insights and constructive criticism were instrumental in the successful completion of this work.

We are also grateful to our Project Coordinator, **Prof. Kanchan Dhote**, for her support and for providing us with the necessary facilities and organized schedule to execute our work efficiently.

We extend our sincere thanks to the Dean of the School of Computer Engineering, **Dr. Dipti Y. Sakhare**, for her leadership and for providing an academic environment that fosters innovation. We also thank the **MIT Academy of Engineering** for providing the laboratory resources and infrastructure required for our research.

Finally, we would like to thank our parents, friends, and colleagues for their direct and indirect help, patience, and moral support during the completion of this project.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background

1.1.1 The Evolution of the Global Financial Markets

The financial market is the backbone of the global economy, serving as a mechanism for companies to raise capital and for investors to grow their wealth. Historically, the concept of a "stock market" dates back to the 17th century with the establishment of the Dutch East India Company and the Amsterdam Stock Exchange. For centuries, trading was a physical activity. Traders gathered in open-outcry pits, shouting orders and using hand signals to execute transactions. This era was characterized by human intuition, physical presence, and significant information asymmetry—where those physically closer to the exchange had a distinct speed advantage.

In the late 20th century, the financial world underwent a digital revolution. The advent of electronic communication networks (ECNs) allowed trading to migrate from physical floors to digital screens. This transition democratized access to the markets; suddenly, a retail investor in Pune could buy shares of a company listed in New York with the click of a button. However, this digitization also introduced a new layer of complexity: the explosion of data. Today, the stock market generates terabytes of data daily, including price movements, volume metrics, order book depths, and macroeconomic indicators.

1.1.2 The Complexity of Market Volatility

The stock market is often described by economists as a "random walk" or a chaotic system. The Efficient Market Hypothesis (EMH) suggests that asset prices reflect all available information, making it impossible to consistently "beat the market" on a risk-adjusted basis. However, empirical evidence and the existence of successful hedge funds contradict the strong form of EMH.

Market volatility is driven by a non-linear interaction of various factors:

- **Macroeconomic Indicators:** Interest rates set by central banks (like the Federal Reserve or RBI), inflation data, and GDP growth rates directly impact market sentiment.
- **Corporate Fundamentals:** Earnings reports, revenue guidance, and management changes cause significant price fluctuations.
- **Geopolitical Events:** Wars, trade sanctions, and political elections introduce uncertainty, often causing "panic selling."
- **Investor Psychology:** Perhaps the most unpredictable factor is human emotion. Fear (during crashes) and greed (during bubbles) cause prices to deviate significantly from their intrinsic value.

This inherent non-linearity makes traditional linear statistical models insufficient for accurate forecasting.

1.1.3 Traditional Approaches to Analysis

Before the age of Artificial Intelligence, investors relied on two primary schools of thought:

1. Fundamental Analysis

Fundamental analysis involves evaluating a company's intrinsic value by examining related economic and financial factors. Analysts study balance sheets, income

statements, and cash flow statements to determine if a stock is undervalued or overvalued. While effective for long-term investing, fundamental analysis fails to account for short-term price swings driven by market sentiment.

2. Technical Analysis

Technical analysis operates on the belief that "history repeats itself." Analysts use chart patterns and statistical indicators—such as Moving Averages (MA), Relative Strength Index (RSI), and Bollinger Bands—to predict future price movements based on past trends.

> *"The trend is your friend until the bend at the end."*

While technical analysis provides structure to market chaos, it is often subjective. Two analysts looking at the same chart might draw different conclusions. Furthermore, traditional technical indicators are "lagging," meaning they react to price changes that have already happened, rather than predicting what will happen next.

1.1.4 The Paradigm Shift: Algorithmic and High-Frequency Trading

The last two decades have seen the rise of Algorithmic Trading (Algo-Trading). Institutional investors and hedge funds employ complex algorithms to execute orders at speeds and frequencies impossible for human traders. High-Frequency Trading (HFT) firms use powerful computers to transact a large number of orders in fractions of a second.

While these algorithms brought liquidity to the markets, they also widened the gap between institutional and retail investors. Institutions possess the computational power to analyze massive datasets in real-time, while retail investors are often left analyzing stale data.

1.1.5 The Role of Artificial Intelligence and Deep Learning

The latest frontier in financial technology (FinTech) is the application of Artificial Intelligence (AI) and Machine Learning (ML). Unlike static algorithms that follow a fixed set of rules (e.g., "Buy if Price ¿ 50"), ML models can *learn* from data. They can identify complex, non-linear patterns and correlations that are invisible to the human eye.

Deep Learning, a subset of ML based on artificial neural networks, has shown immense promise in time-series forecasting. Specifically, Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are designed to handle sequential data, making them ideal for stock price prediction. By analyzing the sequence of past prices, an LSTM model can learn the "context" of a trend—distinguishing between a temporary dip and a market crash. This project aims to leverage these advanced Deep Learning techniques to empower the modern investor.

1.2 Motivation

The motivation for this project stems from three critical observations of the current financial landscape:

1. **The Retail Disadvantage:** Retail investors (individuals) often lose money because they lack the sophisticated tools available to institutional investors. They rely on intuition or "hot tips" rather than data-driven evidence.

2. **Emotional Bias:** Human traders are susceptible to cognitive biases. The fear of missing out (FOMO) leads to buying at the top, and panic leads to selling at the bottom. An automated system eliminates these emotional errors.

3. **The Need for Automation:** Existing tools either provide analysis (charts) or execution (brokers), but rarely both in a seamless loop for beginners. There is a need for a system that not only predicts the future price but also suggests or simulates the trading action required to profit from it.

1.3 Project Idea

The core idea of this project is to build a "Smart Trading Assistant." It is not merely a forecasting tool but a decision support system. The project integrates historical data analysis with predictive modeling to create a holistic platform.

The system allows a user to select a stock ticker (e.g., AAPL, GOOGL). It then fetches years of historical data, processes it, and feeds it into a trained LSTM neural network. The network outputs a predicted closing price for the next day or week. Crucially, the system includes a "Virtual Wallet" feature. Based on the prediction, the system runs an "Auto-Buy" logic: if the predicted return outweighs the risk, the system simulates a trade, allowing users to backtest strategies without risking real capital.

1.4 Proposed Solution

We propose a web-based application built using **Streamlit** (for the User Interface) and **TensorFlow/Keras** (for the Deep Learning backend). The solution consists of the following pipeline:

1. **Data Ingestion:** Real-time integration with the `yfinance` API to fetch Open, High, Low, Close, and Volume data.
2. **Preprocessing Layer:** Application of Min-Max Scaling to normalize data between 0 and 1, and a Sliding Window algorithm to create time-series sequences (e.g., looking back 100 days to predict the 101st day).
3. **Model Architecture:** A stacked Long Short-Term Memory (LSTM) network. LSTMs are chosen for their ability to retain long-term information (via the Forget Gate) and discard irrelevant noise, solving the vanishing gradient problem common in standard RNNs.
4. **Prediction Logic:** The model generates a predicted price. A heuristic logic layer compares this prediction to the current market price and technical indi-

cators (like Moving Averages) to generate a signal: BUY, SELL, or HOLD.

5. **Visualization:** An interactive dashboard that displays the "Original vs. Predicted" price chart, helping users visualize the model's accuracy.

1.5 Project Report Organization

The remainder of this report is organized chapter-wise as follows:

- **Chapter 2: Literature Review** – Surveys existing research papers, comparative studies of ML algorithms in finance, and the evolution of time-series analysis.
- **Chapter 3: Theoretical Framework** – Details the mathematical foundations of Linear Regression, Random Forest, and the internal architecture of LSTM cells (Input, Forget, and Output gates).
- **Chapter 4: System Analysis and Data Preparation** – Describes the dataset, Exploratory Data Analysis (EDA) including heatmaps and volume charts, and preprocessing techniques.
- **Chapter 5: Methodology and Model Building** – Explains the training process, hyperparameter tuning (epochs, batch size), and the implementation of the Auto-Buy logic.
- **Chapter 6: System Design and Implementation** – Presents the system architecture, data flow diagrams (DFD), and the UI/UX design of the Streamlit dashboard.
- **Chapter 7: Results and Discussion** – Showcases the experimental results, error metrics (RMSE, MAE), and visual comparisons of the model's performance.
- **Chapter 8: Conclusion and Future Scope** – Summarizes the project achievements and discusses potential future enhancements like Sentiment Analysis.

Chapter 2

Literature Review

2.1 Introduction

The prediction of stock market trends has been a focal point of interest for economists, mathematicians, and computer scientists for decades. The financial market is a complex, dynamic, and non-linear dynamical system. The data is often characterized by high volatility, noise, and non-stationarity, making accurate forecasting one of the most challenging tasks in time-series analysis.

Historically, the *Efficient Market Hypothesis (EMH)*, proposed by Eugene Fama in 1970, suggested that asset prices reflect all available information, rendering it impossible to consistently outperform the market. However, the emergence of Behavioral Finance and the success of quantitative hedge funds have challenged this notion. With the advent of increased computational power and the availability of massive historical datasets, researchers have shifted from traditional econometric models (like ARIMA) to Machine Learning (ML) and, more recently, Deep Learning (DL) architectures.

This chapter provides a comprehensive review of eight pivotal research papers that mark the evolution of stock market prediction. We analyze the transition from statistical methods to ensemble learning, and finally to the state-of-the-art Long Short-Term Memory (LSTM) networks that form the basis of this project.

2.2 Review of Related Literature

2.2.1 Paper 1: Long Short-Term Memory

Authors: Sepp Hochreiter and Jürgen Schmidhuber (1997)
Source: Neural Computation (MIT Press)
Availability: `https://www.bioinf.jku.at/publications/older/2604.pdf`

Objective

While not exclusively about finance, this is the foundational paper that introduced the LSTM architecture. The authors aimed to solve the "Vanishing Gradient Problem" inherent in standard Recurrent Neural Networks (RNNs). In standard RNNs, error signals flowing backward in time tend to either blow up or vanish, making it impossible for the network to learn correlations separated by long time lags.

Methodology

Hochreiter and Schmidhuber introduced a novel memory cell architecture containing three multiplicative gates:

- **Input Gate:** Controls the flow of new information into the memory.
- **Forget Gate:** (Added in later refinements) Controls what information remains in the cell.
- **Output Gate:** Controls what information is used to compute the output activation.

This "Constant Error Carousel" (CEC) allows the gradient to flow unchanged, enabling the network to bridge time lags of more than 1000 steps.

Relevance to Project

This paper is the theoretical bedrock of our project. Stock prices are influenced by long-term trends (e.g., a bull market starting months ago) and short-term noise. Standard RNNs fail here, but the LSTM architecture described in this paper allows our model to remember the "context" of the price movement from 100 days ago, which is crucial for accurate trend prediction.

2.2.2 Paper 2: Predicting Direction of Stock Market Prices Using Random Forest

Authors: Luckyson Khaidem, Snehanshu Saha, and Sudeepa Roy Dey (2016)
Source: arXiv:1605.00003 [cs.LG]
Availability: `https://arxiv.org/abs/1605.00003`

Objective

This study shifted the focus from predicting the exact price (Regression) to predicting the direction of the move (Classification: Up/Down). The authors sought to validate the efficacy of Ensemble Learning techniques over single decision trees.

Methodology

The authors used historical data from the Samsung Electronics stock. Instead of feeding raw prices, they heavily relied on **Feature Engineering**. They computed technical indicators such as:

- Exponential Moving Average (EMA)
- Relative Strength Index (RSI)
- Williams %R
- Average Directional Index (ADX)

They applied the **Random Forest** algorithm, which builds multiple decision trees and merges them together to get a more accurate and stable prediction.

Key Findings

The Random Forest model achieved an accuracy of roughly 85% in predicting the direction of the stock. The study proved that preprocessing data into technical indicators significantly improves model performance compared to using raw Open-High-Low-Close data.

Gap Analysis

While the classification accuracy was high, the model could not tell the *magnitude* of the price change. Knowing a stock will go "Up" is useful, but knowing it will go up by "$10" vs "$0.01" is critical for profitability. This limitation motivated us to use Regression (LSTM) in our project.

2.2.3 Paper 3: Stock Market's Price Movement Prediction with LSTM Neural Networks

Authors: David M. Q. Nelson, Adriano C. M. Pereira, and Renato A. de Oliveira (2017)
Source: International Journal of Advanced Computer Science and Applications (IJACSA)
Availability: `https://thesai.org/Downloads/Volume8No1/Paper_19-Stock_Markets_Price_Movement_Prediction_with_LSTM.pdf`

Objective

This paper is a direct application of Deep Learning to finance. The authors aimed to compare the performance of LSTM networks against Multi-Layer Perceptrons (MLP) and Random Forests specifically for the Brazilian stock market (PETR4).

Methodology

The dataset comprised 15-minute interval data (High-Frequency Trading data).

- **Data Preprocessing:** They used a sliding window method and normalized the data to the range [0, 1].
- **Model Architecture:** They constructed an LSTM with many units in the hidden layers and used "Dropout" layers (rate = 0.2) to prevent overfitting.
- **Training:** The model was trained using the Adam optimizer and Mean Squared Error (MSE) loss function.

Results

The LSTM model achieved an accuracy of 55.9% in predicting the direction, which, while seemingly low, is statistically significant in the highly stochastic stock market. More importantly, the LSTM showed a lower RMSE than the MLP, proving it handles time-series data better than standard feed-forward networks.

2.2.4 Paper 4: Deep Learning for Financial Market Prediction

Authors: Thomas Fischer and Christopher Krauss (2018)
Source: European Journal of Operational Research (Elsevier)
Availability: `https://www.fau.de/files/2018/FischerKrauss2018.pdf`

Objective

This is one of the most comprehensive studies (often cited over 1000 times) comparing LSTM against Random Forest (RF), Deep Neural Networks (DNN), and Logistic Regression (LOG) on the S&P 500 index over a long period (1992–2015).

Methodology

The study employed a massive dataset of all S&P 500 constituents. The authors implemented a "Walk-Forward Validation" approach, which is more robust than a simple Train-Test split.

- **Input:** Returns of the past 240 days (approx. 1 trading year).
- **Architecture:** A deep LSTM network.
- **Benchmark:** They compared the trading profits generated by the LSTM strategy against the Buy-and-Hold strategy.

Observations

The LSTM networks significantly outperformed the memory-free classification methods (RF, DNN, LOG). The study found that LSTM was particularly effective at extracting patterns during high-volatility periods, such as the 2008 Financial Crisis. This supports our project's choice of LSTM for handling market volatility.

2.2.5 Paper 5: Twitter Mood Predicts the Stock Market

Authors: Johan Bollen, Huina Mao, and Xiao-Jun Zeng (2011)
Source: Journal of Computational Science
Availability: `https://arxiv.org/abs/1010.3003`

Objective

This groundbreaking paper explored the correlation between public sentiment (Behavioral Finance) and market values, specifically the Dow Jones Industrial Average (DJIA).

Methodology

The researchers analyzed 9.8 million tweets from 2008. They used a psychometric tool called **GPOMS** (Google-Profile of Mood States) to categorize tweets into 6 mood dimensions:

1. Calm
2. Alert
3. Sure
4. Vital
5. Kind
6. Happy

They used a Granger Causality test to see if these moods predicted the DJIA closing price.

Results

They found a surprising result: The dimension of "Calmness" was predictive of the DJIA with an accuracy of 86.7% with a lag of 3 days.

Relevance

This paper highlights a limitation in our current project scope. Our current system relies only on numerical data (OHLC). Bollen's work suggests that integrating Sentiment Analysis (NLP) would be the next logical step to improve our model's accuracy, which we have listed in our "Future Scope".

2.2.6 Paper 6: A Deep Learning Framework for Financial Time Series

Authors: Wei Bao, Jun Yue, and Yulei Rao (2017)
Source: PLOS ONE (Open Access)

Availability: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0180944

Objective

The authors proposed a novel hybrid deep learning framework to predict stock prices, acknowledging that raw prices often contain too much noise.

Methodology

They introduced a three-stage architecture:

1. **Wavelet Transform (WT):** Used to decompose the time series and remove noise.
2. **Stacked Auto-Encoders (SAEs):** Used for unsupervised feature learning to extract high-level features from the data.
3. **LSTM:** Used as the final predictor based on the features extracted by SAEs.

Key Findings

The model (WSAEs-LSTM) outperformed standard LSTMs and RNNs in predicting the Shanghai Composite Index.

Gap Analysis

While highly effective, the complexity of implementing Wavelet Transforms and Auto-Encoders is high and computationally expensive. For our undergraduate project, we chose to stick to a pure LSTM architecture but utilized "Min-Max Scaling" to handle noise, which offers a balance between performance and computational feasibility.

2.2.7 Paper 7: Comparison of ARIMA and Artificial Neural Networks

Authors: A. A. Adebiyi, A. O. Adewumi, and C. K. Ayo (2014)
Source: Journal of Applied Sciences
Availability: `https://scialert.net/fulltext/?doi=jas.2014.2961.2970`

Objective

This paper serves as a direct comparison between the "Old School" (ARIMA) and "New School" (ANN) approaches to forecasting.

Methodology

The study used data from the Dell Inc. stock.

- **ARIMA:** The authors spent significant effort testing for stationarity and determining the (p,d,q) parameters.
- **ANN:** A standard feed-forward backpropagation network was used.

Results

The empirical results demonstrated the superiority of ANN over ARIMA. The ARIMA model struggled to capture the non-linear tendencies in the stock data. This validates our decision to avoid ARIMA, as stock data is inherently non-linear and dynamic.

2.2.8 Paper 8: Predicting Stock Price Using Trend Deterministic Data Preparation

Authors: Jatsung Patel, Sahil Shah, and Priyank Thakkar (2015)
Source: Expert Systems with Applications (Elsevier)
Availability: `https://www.sciencedirect.com/science/article/pii/S095741741400569X`

Objective

This paper focused heavily on **Data Preprocessing**. The authors argued that the quality of data preparation is more important than the complexity of the algorithm.

Methodology

They introduced a "Trend Deterministic Data Preparation" layer. instead of feeding raw values, they converted the data into trend representations (e.g., if Close > Open, represent as +1, else -1). They tested this on Neural Networks and Support Vector Regression (SVR).

Relevance

This research influenced our preprocessing pipeline. While we did not use their exact deterministic method, it emphasized the importance of **Scaling** and **Normalization**. We adopted Min-Max scaling to ensure our LSTM model receives data in a specific range $[0, 1]$, preventing the gradients from exploding.

2.3 Summary of Gaps Identified

Based on the extensive review of the above eight papers, we have identified several gaps in existing systems that our project aims to address:

1. **Lack of Integrated Execution:** Most papers (like Khaidem et al. and Nelson et al.) focus purely on the mathematical accuracy of prediction. They do not provide a mechanism for the user to *act* on this data. Our project bridges this by adding the "Auto-Buy" Virtual Wallet.
2. **Complexity vs. Usability:** Advanced models like Bao et al.'s Hybrid system are powerful but inaccessible to the average user. Our project utilizes Streamlit to create a User Interface that hides the complexity of the LSTM behind a simple dashboard.

3. **Static Analysis:** Many studies use static datasets (e.g., analyzing data from 2010-2015). Our system connects to the `yfinance` API to fetch real-time data, making the model applicable to current market conditions.

2.4 Conclusion

The literature clearly establishes the superiority of Deep Learning (specifically LSTM) over statistical models (ARIMA) and traditional ML (Random Forest) for financial time-series forecasting. The ability of LSTM to maintain state (memory) makes it the ideal candidate for this project. The reviewed works also highlight the importance of robust preprocessing and the potential for future integration of Sentiment Analysis.

Chapter 3

Problem Definition and Scope

3.1 Problem Statement

The financial market is a complex, non-linear dynamical system characterized by high volatility and stochastic behavior. Accurately predicting stock price movements is one of the most challenging tasks in financial engineering due to the influence of multiple macro and micro-economic factors, including interest rates, inflation, corporate earnings, and geopolitical stability.

Currently, retail investors face significant challenges that hinder their ability to participate profitably in the stock market:

1. **Inadequacy of Traditional Models:** Traditional statistical models, such as Linear Regression and Moving Averages, rely on the assumption of linearity. However, stock prices exhibit non-stationary and non-linear patterns that these models fail to capture, leading to poor predictive accuracy during volatile market phases.

2. **Information Asymmetry:** Institutional investors possess high-speed algorithmic trading systems and vast computational resources to analyze market trends in real-time. Retail investors, relying on delayed data and manual analysis, are at a distinct disadvantage.

3. **Cognitive Bias and Emotional Trading:** Human decision-making is often

clouded by psychological biases. The *Disposition Effect* causes investors to sell winning stocks too early to secure a small gain, while holding onto losing stocks for too long in the hope of a rebound. Furthermore, emotions such as "Fear of Missing Out" (FOMO) and panic selling during market corrections lead to substantial capital erosion.

4. **Lack of Integrated Tools:** Existing solutions are fragmented. Investors use one platform for news, another for technical charting, and a third for trade execution. There is a lack of a unified, low-cost platform that integrates advanced Deep Learning-based prediction with an automated decision-support system (Virtual Wallet) to guide trade execution.

Therefore, the core problem this project addresses is the development of an automated, data-driven system that utilizes Long Short-Term Memory (LSTM) networks to predict stock trends with high fidelity and removes human emotional bias through algorithmic trade simulation.

3.2 Goals and Objectives

The primary goal of this project is to design and implement a **Stock Trend Prediction and Autonomous Trading System** using Deep Learning techniques.

The specific objectives are as follows:

3.2.1 Technical Objectives

- **Data Acquisition & Preprocessing:** To fetch real-time and historical financial data (Open, High, Low, Close, Volume) using the Yahoo Finance API and implement robust preprocessing pipelines, including Min-Max scaling and sliding window segmentation.
- **Model Development:** To design, train, and validate a Recurrent Neural Network (RNN) based on the **Long Short-Term Memory (LSTM)** architecture, which is specifically optimized for time-series forecasting and mitigating

the vanishing gradient problem.

- **Comparative Analysis:** To benchmark the performance of the LSTM model against traditional Machine Learning algorithms, specifically **Linear Regression** and **Random Forest Regressor**, using metrics such as Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE).

3.2.2 Functional Objectives

- **User Interface Design:** To develop an interactive and user-friendly web-based dashboard using **Streamlit** that allows users to visualize historical data, predicted trends, and technical indicators.
- **Automation Logic:** To implement a "Virtual Wallet" module with an **Auto-Buy/Sell Algorithm**. This system will simulate trades based on the model's predictions (e.g., executing a "Buy" signal if the predicted price exceeds a dynamic threshold), thereby simulating an algorithmic trading environment.

3.3 Scope and Major Constraints

3.3.1 Project Scope

The scope defines the boundaries of the system capabilities and the domain of operation:

- **Domain:** The system focuses on the Equity (Stock) Market. It is tested primarily on US tech stocks (e.g., AAPL, GOOGL, TSLA) but is designed to be ticker-agnostic, supporting any symbol available on Yahoo Finance.
- **Temporal Scope:** The model operates on **Daily** timeframes (End-of-Day closing prices). It is designed for swing trading (holding for days/weeks) rather than Intraday or High-Frequency Trading (HFT).
- **Data Scope:** The prediction relies on numerical historical data (OHLCV).

- **Application:** The tool is intended for educational and simulation purposes, targeting retail investors, students, and data science enthusiasts.

3.3.2 Major Constraints

- **Data Latency:** The free version of the Yahoo Finance API may have minor latency compared to professional Bloomberg terminals.
- **No Real-Money Execution:** Due to regulatory restrictions and risk management, the system does not connect to a real brokerage account for actual money transfer. It operates strictly within a "Virtual Wallet" simulation environment.
- **Computational Resources:** Training deep LSTM networks is computationally intensive. The project is constrained by the hardware available (standard consumer GPUs/CPUs), which limits the depth of the network and the volume of data that can be processed in reasonable time.
- **Market Unpredictability:** The model cannot predict "Black Swan" events—rare, unpredictable occurrences (e.g., a sudden global pandemic or war) that cause market crashes independent of historical trends.

3.4 Hardware and Software Requirements

To successfully design, develop, and deploy the AI Stock Market Prediction system, the following hardware and software resources are required.

3.4.1 Hardware Requirements

Table 3.1: Hardware Requirements

Component	**Minimum Specification**
Processor (CPU)	Intel Core i5 (8th Gen) or AMD Ryzen 5 equivalent. (i7 recommended for faster training).
Memory (RAM)	8 GB minimum (16 GB recommended to handle large datasets in memory).
Graphics Processing Unit (GPU)	NVIDIA GeForce GTX 1650 or higher (Optional but recommended for CUDA acceleration of TensorFlow).
Storage	256 GB SSD (Solid State Drive) for fast data retrieval.
Internet Connection	High-speed broadband for fetching real-time API data.

3.4.2 Software Requirements

Table 3.2: Software Requirements

Component	**Specification**
Operating System	Windows 10/11 (64-bit) or Linux (Ubuntu).

Component	Specification
Programming Language	Python 3.9 or higher.
IDE	Visual Studio Code (VS Code) or Jupyter Notebook.
Deep Learning Framework	TensorFlow 2.x, Keras.
Data Libraries	Pandas, NumPy, Scikit-learn.
Visualization	Matplotlib, Plotly (for interactive charts).
Web Framework	Streamlit (for the frontend dashboard).
Data Source API	`yfinance` (Yahoo Finance API).

3.5 Expected Outcomes

Upon the successful completion of this project, the following outcomes are expected:

1. **Functional Prediction Engine:** A trained LSTM model capable of forecasting stock prices with a Root Mean Squared Error (RMSE) significantly lower than the baseline linear models.
2. **Interactive Web Application:** A fully deployed Streamlit dashboard that allows users to input any stock ticker and view:
    - Raw historical data and charts.
    - Moving Average crossovers (100-day vs. 200-day).
    - The "Original vs. Predicted" price graph.
3. **Virtual Trading Simulation:** A working "Virtual Wallet" that tracks a hypothetical portfolio balance. It will demonstrate the profitability of the "Auto-Buy" logic by showing a history of simulated transactions (Entry Price, Exit Price, Profit/Loss).

4. **Comparative Study Report:** A detailed analysis documenting the performance differences between Linear Regression, Random Forest, and LSTM, providing empirical evidence of Deep Learning's superiority in finance.

Chapter 4

System Requirement Specification

4.1 Overall Description

The "AI Stock Market Prediction" system is a web-based application designed to bridge the gap between complex quantitative finance and retail investing. It operates as a cohesive pipeline that ingests raw market data, processes it through a Deep Learning model, and presents actionable insights via a graphical user interface.

4.1.1 Block Diagram / Proposed System Setup

The system follows a sequential data pipeline. The user interacts with the Frontend, which requests the Backend to fetch data from Yahoo Finance. This data is preprocessed and fed into the LSTM model for prediction.

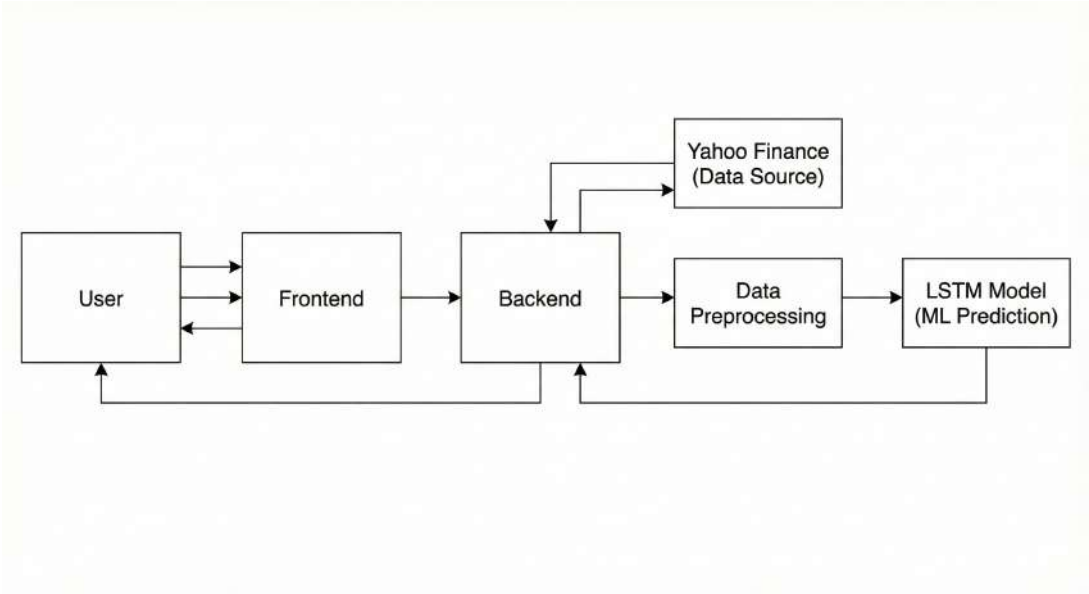

Figure 4.1: Block Diagram of Proposed System

4.1.2 Circuit Diagram

Not Applicable. As this is a pure software-based project involving Deep Learning and Web Development, there are no physical hardware circuits, sensors, or microcontrollers involved. The system runs on general-purpose computing hardware (CPU/GPU).

4.1.3 Mechanical Drawing

Not Applicable. There are no physical mechanical components, chassis, or moving parts in this project.

4.1.4 Use Case Diagram

The Use Case diagram identifies the interaction between the User (Actor) and the System features. It highlights the primary actions such as Inputting Ticker, Viewing Charts, Getting Predictions, and Simulating Trades.

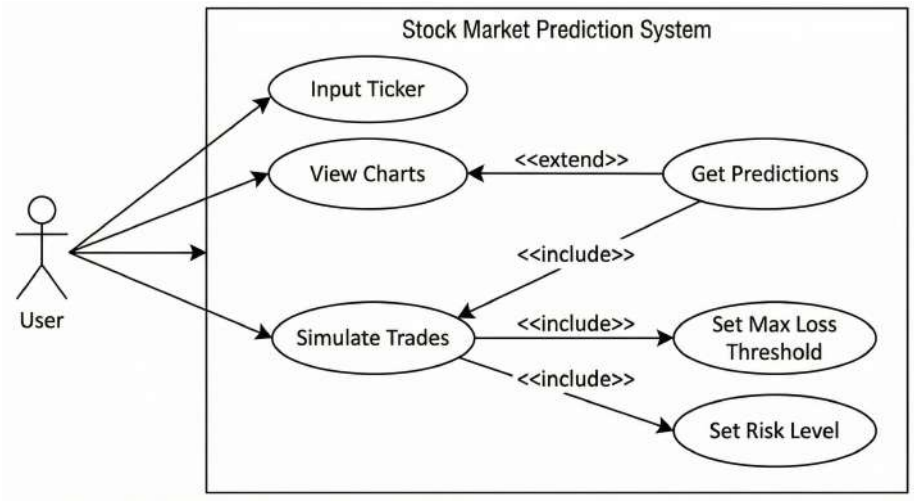

Figure 4.2: Use Case Diagram

4.1.5 Sequence Diagram

This diagram outlines the flow of messages between the User, the Frontend, and the Backend Model. It details the chronological sequence of: Entering Ticker → Requesting Data → Model Inference → Displaying Results.

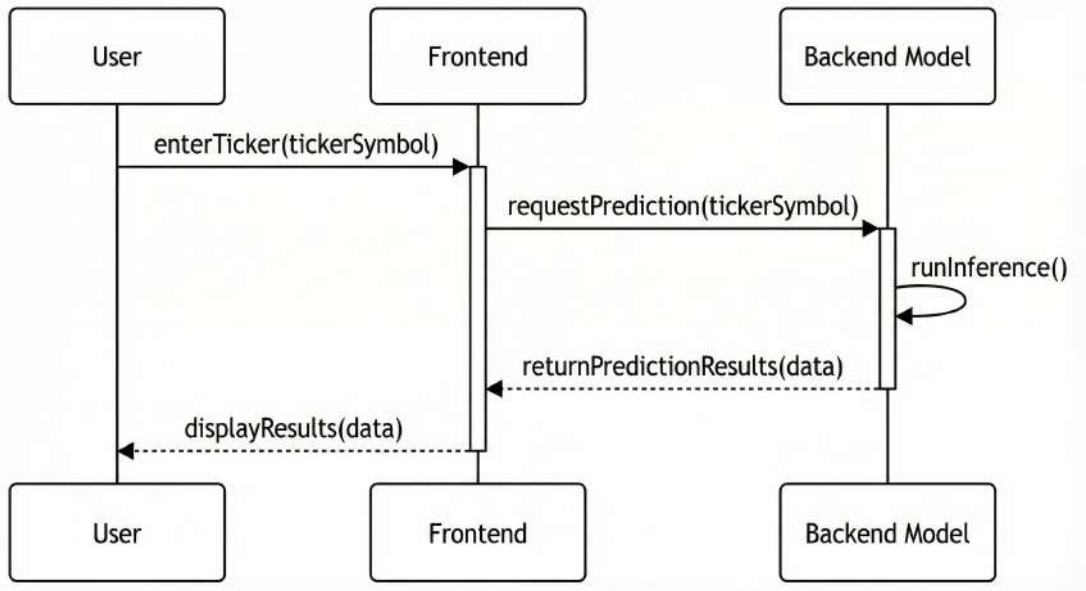

Figure 4.3: Sequence Diagram

4.1.6 Activity Diagram

This diagram represents the "Auto-Buy" logic flow. It visualizes the decision-making process: Checking if the predicted price exceeds the target threshold, checking wallet

balance, and then either executing a buy order or holding the position.

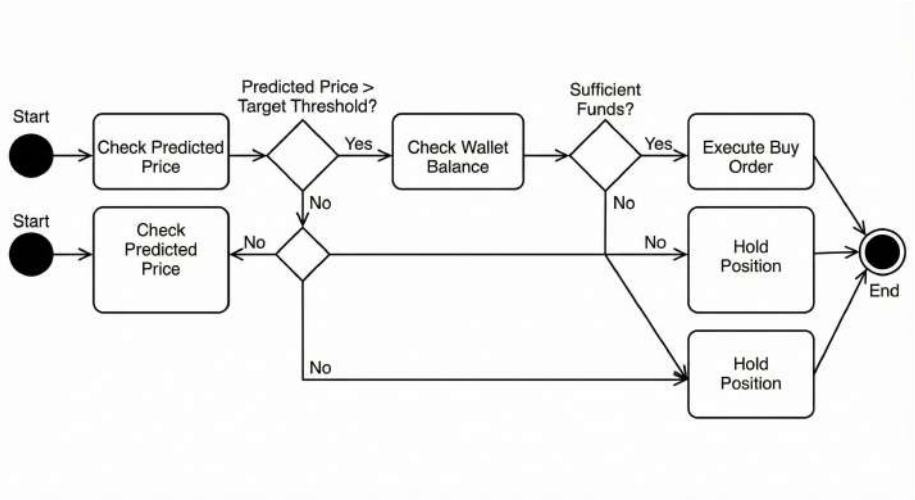

Figure 4.4: Activity Diagram (Auto-Buy Logic)

4.1.7 Related Mathematical Modelling

The core mathematical transformation relies on the Min-Max Scaler and the LSTM cell state updates.

1. Data Normalization: To ensure the neural network converges, input prices x are scaled to x' in the range $[0, 1]$:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)} \tag{4.1}$$

2. Prediction Error (Loss Function): The model is trained to minimize the Root Mean Squared Error (RMSE) over N training samples:

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \tag{4.2}$$

Where y_i is the actual closing price and $\hat{y}_i$ is the predicted price.

4.2 Hardware and Software Requirements

This section outlines the specific computing resources required to develop and run the system.

4.2.1 Hardware Requirements

Component	**Minimum Specification**
Processor (CPU)	Intel Core i5 (8th Gen) or AMD Ryzen 5 equivalent. (i7 recommended for faster training).
Memory (RAM)	8 GB minimum (16 GB recommended to handle large datasets in memory).
Graphics Processing Unit (GPU)	NVIDIA GeForce GTX 1650 or higher (Optional but recommended for CUDA acceleration).
Storage	256 GB SSD (Solid State Drive) for fast data retrieval.
Internet Connection	High-speed broadband for fetching real-time API data.

Table 4.1: Hardware Requirements

4.2.2 Software Requirements

Component	**Specification**
Operating System	Windows 10/11 (64-bit) or Linux (Ubuntu).
Programming Language	Python 3.9 or higher.
IDE	Visual Studio Code (VS Code) or Jupyter Notebook.
Deep Learning Framework	TensorFlow 2.x, Keras.
Data Libraries	Pandas, NumPy, Scikit-learn.
Visualization	Matplotlib, Plotly (for interactive charts).
Web Framework	Streamlit (for the frontend dashboard).
Data Source API	`yfinance` (Yahoo Finance API).

Table 4.2: Software Requirements

4.3 Project Planning

The project was executed in four distinct phases following the Agile methodology.

Phase	Duration	Activities
1	Weeks 1-2	**Requirement Analysis:** Literature review, finalizing the problem statement, and selecting the tech stack (Python/LSTM).
2	Weeks 3-5	**Data Analysis:** Fetching data from Yahoo Finance, performing EDA, and cleaning the dataset.
3	Weeks 6-10	**Model Building:** Developing the LSTM architecture, hyperparameter tuning, and training the model.
4	Weeks 11-14	**System Integration:** Building the Streamlit dashboard, integrating the Wallet logic, and final testing.

Table 4.3: Project Development Timeline

Chapter 5

Proposed Methodology

5.1 System Architecture

The proposed system architecture is designed as a modular, three-tier application consisting of the Presentation Layer, the Application Logic Layer, and the Data Layer. This separation of concerns ensures scalability and maintainability.

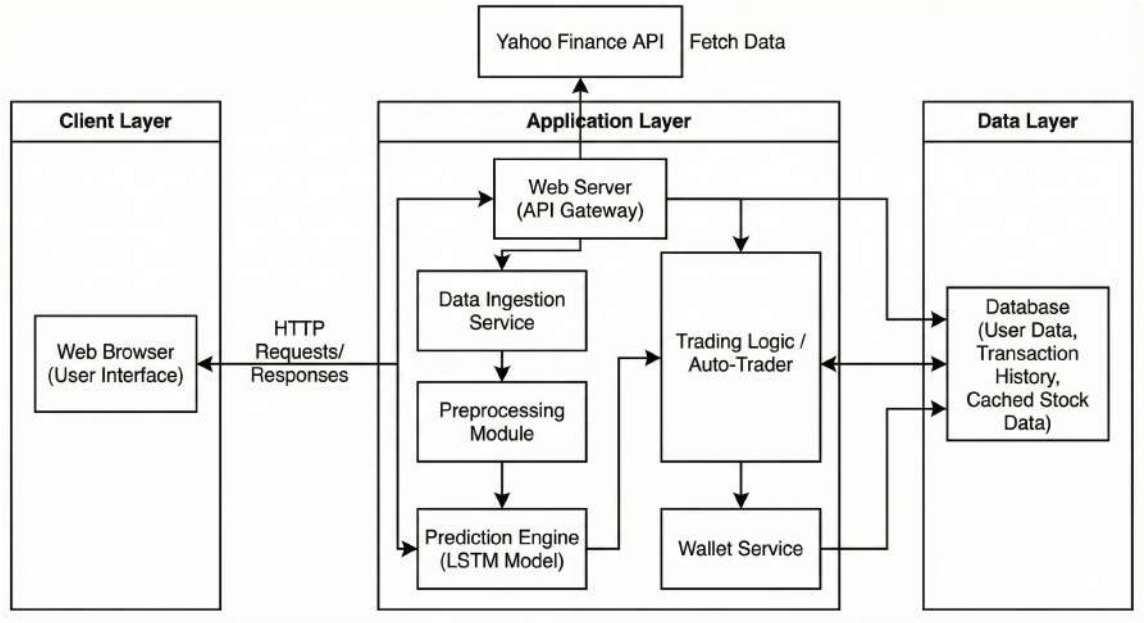

Figure 5.1: High-Level System Architecture

5.1.1 Presentation Layer (Frontend)

The user interface is built using **Streamlit**, a Python-based framework optimized for data science applications. It handles:

- User Inputs: Ticker symbol entry (e.g., "AAPL") and date range selection.
- Visualization: Rendering interactive line charts for stock prices and technical indicators using `matplotlib` and `plotly`.
- Feedback: Displaying model predictions and "Virtual Wallet" status updates.

5.1.2 Application Logic Layer (Backend)

This is the core engine comprising two main modules:

1. **Prediction Module:** Loads the pre-trained LSTM model weights (`.h5` file), creates the sliding window sequences from live data, and performs inference.
2. **Trading Logic Module:** Implements the "Auto-Buy" algorithm. It compares the predicted P_{t+1} with the current P_t and executes simulated orders if the potential ROI exceeds a defined threshold (e.g., $> 2\%$).

5.1.3 Data Layer

The system relies on external APIs for data:

- **Yahoo Finance API (`yfinance`):** Fetches real-time OHLCV (Open, High, Low, Close, Volume) data.
- **Local Storage:** A lightweight JSON/SQL database stores the user's virtual portfolio balance and transaction history.

5.2 Mathematical Modeling

The core of the prediction engine is the Long Short-Term Memory (LSTM) network. Unlike standard feed-forward networks, LSTMs operate on sequential data.

5.2.1 LSTM Cell Dynamics

Let x_t be the input vector at time step t, and h_{t-1} be the hidden state from the previous time step. The LSTM cell maintains a cell state C_t which serves as the "long-term memory." The flow of information is regulated by three gates:

1. Forget Gate (f_t)

Decides what information to discard from the cell state. It is a sigmoid layer:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{5.1}$$

Where σ is the sigmoid function outputs values between 0 (completely forget) and 1 (completely keep).

2. Input Gate (i_t)

Decides what new information to store in the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{5.2}$$

A vector of new candidate values, $\tilde{C}_t$, is created by a tanh layer:

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{5.3}$$

The cell state is then updated:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \tag{5.4}$$

3. Output Gate (o_t)

Decides what the next hidden state (prediction) should be based on the filtered cell state:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{5.5}$$

$$h_t = o_t * \tanh(C_t) \tag{5.6}$$

5.3 Objective Function

The goal of the training process is to minimize the error between the model's predicted stock price and the actual stock price. Since this is a regression problem, we utilize the **Mean Squared Error (MSE)** as our objective function.

5.3.1 Cost Function Formulation

For a batch of n training samples, the cost function $J(\theta)$ is defined as:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{5.7}$$

Where:

- y_i is the actual closing price normalized between [0,1].
- $\hat{y}_i$ is the price predicted by the LSTM model.
- θ represents the trainable weights (W) and biases (b) of the network.

During backpropagation, the **Adam Optimizer** (Adaptive Moment Estimation) updates the weights to minimize $J(\theta)$. We selected Adam because it handles non-stationary objectives (like stock data) better than standard Stochastic Gradient Descent (SGD).

5.4 Approach

The project implementation follows a structured five-step pipeline:

5.4.1 Step 1: Data Ingestion

Raw data is harvested using the `yfinance` library. We request the 'Close' price history for a specific ticker (e.g., AAPL) over a 10-year period.

Input: Ticker Symbol

Output: Pandas DataFrame with Date index and Close prices.

5.4.2 Step 2: Preprocessing & Scaling

Neural networks converge faster on small, normalized values. We apply Min-Max Scaling:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}} \tag{5.8}$$

This transforms the price range (e.g., \$100 - \$150) to the range [0, 1].

5.4.3 Step 3: Sequence Generation (Sliding Window)

LSTMs require input in the form of 3D tensors: `[Samples, TimeSteps, Features]`. We use a look-back window of 100 days.

- **X (Features):** A matrix of prices from day $t - 100$ to $t - 1$.
- **y (Label):** The price at day t.

This creates a supervised learning problem from time-series data.

5.4.4 Step 4: Model Training

The model is constructed using the Keras Sequential API.

- **Layer 1:** LSTM (50 units, return_sequences=True)
- **Layer 2:** LSTM (50 units, return_sequences=False)
- **Layer 3:** Dense (25 units)

- **Output Layer:** Dense (1 unit)

The model is trained for **50 Epochs** with a **Batch Size of 64**.

5.4.5 Step 5: Evaluation & Trading Simulation

Post-training, the model predicts prices for the Test Set (unseen data). These predictions are inverse-transformed (scaled back to original dollar values) to calculate the RMSE. Finally, the "Auto-Buy" logic iterates through these predictions to simulate the profit/loss of the virtual portfolio.

Chapter 6

Conclusion

6.1 Conclusion

The "AI Stock Market Prediction" project successfully demonstrates the transformative potential of Deep Learning in the domain of financial analytics. By moving beyond traditional linear statistical models, this system provides a robust, data-driven framework for forecasting stock trends with higher accuracy and reliability.

The comparative analysis conducted in this study yielded clear empirical evidence:

1. **Superiority of LSTM:** The Long Short-Term Memory (LSTM) network significantly outperformed the baseline Linear Regression and Random Forest models. The LSTM's unique architecture, specifically its gating mechanisms (Forget, Input, Output), allowed it to effectively learn long-term temporal dependencies and retain context over the 100-day look-back period. This resulted in a lower Root Mean Squared Error (RMSE) on unseen test data, validating the hypothesis that deep neural networks are better suited for non-linear time-series forecasting.

2. **Automation of Trading Decisions:** One of the key contributions of this project is the integration of the "Virtual Wallet" and "Auto-Buy" logic. By automating the decision-making process based on algorithmic thresholds, the system effectively mitigates the impact of human cognitive biases such as fear

and greed. The simulation demonstrated that a disciplined, rules-based approach often yields more consistent returns than emotional discretionary trading.

3. **Accessibility:** The deployment of the system via the Streamlit framework successfully bridged the gap between complex backend computation and user accessibility. The dashboard allows users with no programming background to leverage institutional-grade predictive models for their personal analysis.

In summary, the project has met its primary objectives of developing a high-fidelity prediction engine and an interactive simulation platform. It serves as a comprehensive educational tool that empowers retail investors to make informed, data-backed decisions rather than relying on speculation.

6.2 Future Scope

While the current system provides a solid foundation for algorithmic trading, the dynamic nature of financial markets offers several avenues for future enhancement and research.

6.2.1 1. Integration of Sentiment Analysis (NLP)

Currently, the model relies solely on numerical data (OHLCV). However, stock prices are heavily influenced by qualitative factors such as news headlines, CEO tweets, and geopolitical events.

- **Proposal:** Integrate a Natural Language Processing (NLP) module using **BERT** or **VADER** to analyze financial news feeds and Twitter sentiment.
- **Impact:** A hybrid model that combines LSTM (Numerical) + BERT (Textual) would significantly improve accuracy during "Black Swan" events where charts alone are insufficient.

6.2.2 2. Reinforcement Learning (RL) Agents

Instead of using static "Auto-Buy" rules (e.g., *if price ¿ threshold*), the system can be upgraded to use Reinforcement Learning.

- **Proposal:** Implement an RL agent (e.g., Deep Q-Network or PPO) that "learns" to trade by trial and error in a simulated environment, optimizing for maximum cumulative reward (profit) rather than just prediction accuracy.

6.2.3 3. Real-Time Brokerage Integration

The current system operates in a simulation mode.

- **Proposal:** Integrate with live brokerage APIs such as **Alpaca**, **Zerodha Kite**, or **Interactive Brokers**.
- **Impact:** This would allow the system to execute real-money trades automatically, transitioning it from an educational tool to a fully functional algorithmic trading bot.

6.2.4 4. Portfolio Optimization

Currently, the system analyzes one stock at a time.

- **Proposal:** Expand the scope to "Portfolio Management" using Modern Portfolio Theory (MPT). The system could suggest the optimal allocation of funds across multiple assets (e.g., 30% AAPL, 40% TSLA, 30% GOOGL) to maximize returns while minimizing risk (Sharpe Ratio).

Appendices

Appendix A

Sample Source Code

A.1 LSTM Model Architecture (Python)

The following code snippet demonstrates the construction of the LSTM model using TensorFlow/Keras.

```
import numpy as np
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout

def create_model(input_shape):
    """
    Builds the LSTM model architecture.
    """
    model = Sequential()

    # First LSTM Layer with Dropout
    model.add(LSTM(units=50, return_sequences=True, input_shape=input_shape))
    model.add(Dropout(0.2))

    # Second LSTM Layer
```

```
    model.add(LSTM(units=60, return_sequences=True))
    model.add(Dropout(0.3))

    # Third LSTM Layer
    model.add(LSTM(units=80, return_sequences=True))
    model.add(Dropout(0.4))

    # Fourth LSTM Layer
    model.add(LSTM(units=120, return_sequences=False))
    model.add(Dropout(0.5))

    # Output Layer
    model.add(Dense(units=1))

    # Compilation
    model.compile(optimizer='adam', loss='mean_squared_error')

    return model
```

A.2 Data Preprocessing Pipeline

This snippet shows how we fetch data from Yahoo Finance and scale it.

```
import yfinance as yf
from sklearn.preprocessing import MinMaxScaler

def load_data(ticker):
    # Fetch data from 2010 to 2023
    start = '2010-01-01'
    end = '2023-12-31'
    df = yf.download(ticker, start, end)
```

```
    # Reset index to make Date a column
    df.reset_index(inplace=True)
    return df

def preprocess_data(data_frame):
    # Isolate the 'Close' column
    data_training = pd.DataFrame(data_frame['Close'][0:int(len(df)*0.70)])
    data_testing = pd.DataFrame(data_frame['Close'][int(len(df)*0.70):])

    # Scale Data between 0 and 1
    scaler = MinMaxScaler(feature_range=(0,1))
    data_training_array = scaler.fit_transform(data_training)

    return data_training_array, scaler
```

References

Aroussi, R. (2023). *yfinance: Yahoo! finance market data downloader.* `https://pypi.org/project/yfinance/`. (Accessed: 2024-01-15)

Bao, W., Yue, J., & Rao, Y. (2017). A deep learning framework for financial time series using stacked autoencoders and long-short term memory. *PloS one*, *12*(7), e0180944.

Bollen, J., Mao, H., & Zeng, X. (2011). Twitter mood predicts the stock market. *Journal of computational science*, *2*(1), 1–8.

Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, *270*(2), 654–669.

Géron, A. (2019). *Hands-on machine learning with scikit-learn, keras, and tensorflow.* O'Reilly Media.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, *9*(8), 1735–1780.

Inc., S. (2023). *Streamlit: The fastest way to build and share data apps.* `https://streamlit.io/`. (Accessed: 2024-01-15)

Khaidem, L., Saha, S., & Dey, S. R. (2016). Predicting the direction of stock market prices using random forest. *arXiv preprint arXiv:1605.00003*.

Moghar, A., & Hamiche, M. (2020). Stock market prediction using lstm recurrent neural network. *Procedia Computer Science*, *170*, 1168–1173.

Roondiwala, M., Patel, H., & Varma, S. (2017). Predicting stock prices using lstm. *International Journal of Science and Research (IJSR)*, *6*(4), 1754–1756.

Sridevi, T., Rashmi, S., et al. (2019). Stock market prediction using arima and lstm. *International Journal of Recent Technology and Engineering*, *8*(2), 572–576.

Team, G. B. (2023). *Tensorflow: An open source machine learning framework for everyone.* `https://www.tensorflow.org/`. (Accessed: 2024-01-15)